

Barrett Viator
DSE6210
Professor Lowhorn
October 18, 2024

Project 2

Part 1 – Data Model

```
1  """
2  Created on Thurs Oct 17 2024
3  DSE6210 Project 2
4
5  @author: Barrett J. Viator
6  """
7
8
9  from pymongo import MongoClient
10
11  # Connect to MongoDB
12  client = MongoClient('mongodb+srv://viatorb:Learningmongodb@cluster0.sz2tw.mongodb.net/')
13
14  # Name the new database and the new collection name
15  project_2_database = 'Airline'
16  agent_collection = 'travel agent'
17  itin_res_collection = 'itinerary reservations'
18  res_payment_collection = 'reservation payments'
19  pass_collection = 'passengers'
20  itin_leg_collection = 'itinerary legs'
21  flight_legs_collection = 'flight legs'
22  airport_collection = 'airports'
23  flight_sched_collection = 'flight schedule'
24  ref_cal_collection = 'reference calendar'
25  flight_cost_collection = "flight cost"
26
27  # Create a new database
28  new_database = client[project_2_database]
29
30  db = client['Airline']
31
32  # Create a new collection
33  agent_col = new_database[agent_collection]
34  res_pay_col = new_database[res_payment_collection]
35  pass_col = new_database[pass_collection]
36  itiner_leg_col = new_database[itiner_leg_collection]
37  itin_res_col = new_database[itin_res_collection]
38  flight_leg_col = new_database[flight_legs_collection]
39  airport_col = new_database[airport_collection]
40  flight_cost_col = new_database[flight_cost_collection]
41  flight_sched_col = new_database[flight_sched_collection]
42  ref_cal_col = new_database[ref_cal_collection]
43
```

```
# booking agents collection

travel_agents = [
    {
        "_id": "045231",
        "agent_name": "Sue Jones",
        "agent_details": "Associate"
    },
    {
        "_id": "067456",
        "agent_name": "Brad Collins",
        "agent_details": "VP"
    },
    {
        "_id": "036524",
        "agent_name": "Ann Rice",
        "agent_details": "Associate"
    }
]
inserted_agent_data = agent_col.insert_many(travel_agents)
```

```

67 #itinerary reservations collection
68
69 itin_res = [
70     {
71         "_id": "15457568",
72         "agent_id": "045231",
73         "passenger_id": "7463826",
74         "reservation_status_code": "01",
75         "payment_amount": 982,
76         "usual_aircraft_type_code": "B747",
77         "origin_airport_code": "LFT",
78         "destination_airport_code": "JFK",
79         "ticket_type_code": "01",
80         "travel_class_code": "75",
81         "date_reservation_made": "2023-10-12",
82         "number_in_party": 1
83     },
84     {
85         "_id": "24668346",
86         "agent_id": "067456",
87         "passenger_id": "5885922",
88         "reservation_status_code": "01",
89         "payment_amount": 473,
90         "usual_aircraft_type_code": "B360",
91         "origin_airport_code": "LFT",
92         "destination_airport_code": "DFW",
93         "ticket_type_code": "01",
94         "travel_class_code": "75",
95         "date_reservation_made": "2023-10-12",
96         "number_in_party": 1
97     },
98     {
99         "_id": "35635773",
100        "agent_id": "045231",
101        "passenger_id": "9284293",
102        "reservation_status_code": "01",
103        "payment_amount": 564,
104        "usual_aircraft_type_code": "564",
105        "origin_airport_code": "LFT",
106        "destination_airport_code": "JFK",
107        "ticket_type_code": "01",
108        "travel_class_code": "75",
109        "date_reservation_made": "2023-10-12",
110        "number_in_party": 1
111    },
112    {
113        "_id": "36725613",
114        "agent_id": "036524",
115        "passenger_id": "7375758",
116        "reservation_status_code": "01",
117        "payment_amount": 638,
118        "usual_aircraft_type_code": "638",
119        "origin_airport_code": "MSY",
120        "destination_airport_code": "ATL",
121        "ticket_type_code": "01",
122        "travel_class_code": "75",
123        "date_reservation_made": "2023-10-12",
124        "number_in_party": 1
125    },
126    {
127        "_id": "74677424",
128        "agent_id": "045231",
129        "passenger_id": "2772849",
130        "reservation_status_code": "01",
131        "payment_amount": 342,
132        "usual_aircraft_type_code": "342",
133        "origin_airport_code": "LFT",
134        "destination_airport_code": "JFK",
135        "ticket_type_code": "01",
136        "travel_class_code": "75",
137        "date_reservation_made": "2023-10-12",
138        "number_in_party": 1
139    }
140 ]
141
142
143 inserted_itin_res_data = itin_res_col.insert_many(itin_res)
144

```

```

146 # reservation payments collection
147
148 res_pay = [
149     {
150         "_id": "AV6345",
151         "pass_id": "7463826",
152         "flight_number": "AA321",
153         "reservation_id": "15457568",
154         "payment_status_code": "05",
155         "payment_date": "10-12-2023",
156         "payment_amount": 982
157     },
158     {
159         "_id": "SD3267",
160         "pass_id": "5885922",
161         "flight_number": "AA321",
162         "reservation_id": "24668346",
163         "payment_status_code": "05",
164         "payment_date": "02-04-2024",
165         "payment_amount": 473
166     },
167     {
168         "_id": "DF3585",
169         "pass_id": "9284293",
170         "flight_number": "AA985",
171         "reservation_id": "35635773",
172         "payment_status_code": "05",
173         "payment_date": "05-11-2024",
174         "payment_amount": 564
175     },
176     {
177         "_id": "LP3563",
178         "pass_id": "7375758",
179         "flight_number": "AA642",
180         "reservation_id": "36725613",
181         "payment_status_code": "05",
182         "payment_date": "08-06-2023",
183         "payment_amount": 638
184     },
185     {
186         "_id": "RF2573",
187         "pass_id": "2772849",
188         "flight_number": "AA321",
189         "reservation_id": "74677424",
190         "payment_status_code": "05",
191         "payment_date": "09-05-2024",
192         "payment_amount": 342
193     }
194 ]
195
196
197 inserted_res_payment_data = res_pay_col.insert_many(res_pay)
198

```

```

199 # passenger collection
200
201 pass_collection = [
202     {
203         "_id": "7463826",
204         "first_name": "Mila",
205         "second_name": "Dina",
206         "last_name": "Romanova",
207         "phone_number": "19287256",
208         "email_address": "mdr@live.com",
209         "address_lines": "2782 1st St",
210         "city": "New York",
211         "state_province_county": "NY",
212         "country": "USA",
213         "other_passenger_details": "Member"
214     },
215     {
216         "_id": "5885922",
217         "first_name": "Laura",
218         "second_name": "Kelly",
219         "last_name": "Juarez",
220         "phone_number": "17389390",
221         "email_address": "Lkj25@gmail.com",
222         "address_lines": "1080 Tree Cove",
223         "city": "Dallas",
224         "state_province_county": "TX",
225         "country": "USA",
226         "other_passenger_details": "Gold Status"
227     },
228     {
229         "_id": "9284293",
230         "first_name": "Frank",
231         "second_name": "Will",
232         "last_name": "Craft",
233         "phone_number": "18373845",
234         "email_address": "fwk6@apple.com",
235         "address_lines": "25 Smith St",
236         "city": "Tempe",
237         "state_province_county": "AZ",
238         "country": "USA",
239         "other_passenger_details": "guest checkout"
240     },
241     {
242         "_id": "7375758",
243         "first_name": "Jian",
244         "second_name": "Li",
245         "last_name": "Cho",
246         "phone_number": "12384658",
247         "email_address": "jlc@yahoo.com",
248         "address_lines": "9876 Rue Verdun",
249         "city": "Montreal",
250         "state_province_county": "CAN",
251         "country": "Canada",
252         "other_passenger_details": "Member"
253     },
254     {
255         "_id": "2772849",
256         "first_name": "Joe",
257         "second_name": "Preston",
258         "last_name": "Hue",
259         "phone_number": "13458293",
260         "email_address": "jph@windows.com",
261         "address_lines": "47 Turnpike Lane",
262         "city": "LA",
263         "state_province_county": "USA",
264         "country": "USA",
265         "other_passenger_details": "Gold Status"

```

```

272 # itinerary legs collection
273
274 itin_leg = [
275     {
276         "_id": {
277             "reservation_id": "15457568",
278             "Leg_id": "056432"
279         },
280         "flight_number": "AA321",
281         "airline_code": "LFT",
282         "origin_airport_code": "LFT",
283         "destination_airport_code": "JFK",
284         "departure_date_time": '2024-01-01 17:00:00',
285         "arrival_date_time": '2024-01-01 20:00:00',
286         "actual_departure_time": '2024-01-01 17:30:00',
287         "actual_arrival_time": '2024-01-01 20:30:00',
288         "departure_efficiency": False
289     },
290     {
291         "_id": {
292             "reservation_id": "24668346",
293             "Leg_id": "056432"
294         },
295         "flight_number": "AA321",
296         "airline_code": "LFT",
297         "origin_airport_code": "MSY",
298         "destination_airport_code": "DFW",
299         "departure_date_time": '2024-01-01 17:00:00',
300         "arrival_date_time": '2024-01-01 20:00:00',
301         "actual_departure_time": '2024-01-01 17:30:00',
302         "actual_arrival_time": '2024-01-01 20:30:00',
303         "departure_efficiency": False
304     },
305     {
306         "_id": {
307             "reservation_id": "35635773",
308             "Leg_id": "656833"
309         },
310         "flight_number": "AA642",
311         "airline_code": "MSY",
312         "origin_airport_code": "LFT",
313         "destination_airport_code": "JFK",
314         "departure_date_time": '2024-01-02 14:00:00',
315         "arrival_date_time": '2024-01-02 15:00:00',
316         "actual_departure_time": '2024-01-02 14:00:00',
317         "actual_arrival_time": '2024-01-02 15:00:00',
318         "departure_efficiency": True
319     },
320     {
321         "_id": {
322             "reservation_id": "36725613",
323             "Leg_id": "345795"
324         },
325         "flight_number": "AA985",
326         "airline_code": "MSY",
327         "origin_airport_code": "MSY",
328         "destination_airport_code": "ATL",
329         "departure_date_time": '2024-01-03 15:00:00',
330         "arrival_date_time": '2024-01-02 17:00:00',
331         "actual_departure_time": '2024-01-03 15:00:00',
332         "actual_arrival_time": '2024-01-02 17:00:00',
333         "departure_efficiency": True
334     },
335     {
336         "_id": {
337             "reservation_id": "74677424",
338             "Leg_id": "056432"
339         },
340         "flight_number": "AA321",
341         "airline_code": "LFT",
342         "origin_airport_code": "LFT",
343         "destination_airport_code": "JFK",
344         "departure_date_time": '2024-01-01 17:00:00',
345         "arrival_date_time": '2024-01-01 20:00:00',
346         "actual_departure_time": '2024-01-01 17:30:00',
347         "actual_arrival_time": '2024-01-01 20:30:00',
348         "departure_efficiency": False
349     }

```

```

350 ]
351
352
353 inserted_itin_leg_data = itiner_leg_col.insert_many(itin_leg)
354
355 # legs collection
356
357 flight_leg = [
358     {
359         "_id": "056432",
360         "flight_number": "AA321",
361         "airline_code": "LFT",
362         "origin_airport_code": "LFT",
363         "destination_airport_code": "JFK",
364         "departure_date_time": "2023-10-12T10:00:00",
365         "arrival_date_time": "2023-10-12T12:00:00"
366     },
367     {
368         "_id": "656833",
369         "flight_number": "AA642",
370         "airline_code": "MSY",
371         "origin_airport_code": "LFT",
372         "destination_airport_code": "DFW",
373         "departure_date_time": "2023-10-12T10:00:00",
374         "arrival_date_time": "2023-10-12T12:00:00"
375     },
376     {
377         "_id": "345795",
378         "flight_number": "AA985",
379         "airline_code": "MSY",
380         "origin_airport_code": "MSY",
381         "destination_airport_code": "ATL",
382         "departure_date_time": "2023-10-12T10:00:00",
383         "arrival_date_time": "2023-10-12T12:00:00"
384     }
385 ]
386
387 inserted_flight_leg_data = flight_leg_col.insert_many(flight_leg)
388
389
390 #airports collection
391 airports = [
392     {
393         "_id": "LFT",
394         "airport_name": "Lafayette Regional",
395         "airport_location": "Lafayette",
396         "other_details": "Regional"
397     },
398     {
399         "_id": "MSY",
400         "airport_name": "Louis Armstrong Airport",
401         "airport_location": "New Orleans",
402         "other_details": "International"
403     },
404     {
405         "_id": "JFK",
406         "airport_name": "John F. Kennedy Airport",
407         "airport_location": "New York City",
408         "other_details": "International"
409     },
410     {
411         "_id": "DFW",
412         "airport_name": "Dallas International Airport",
413         "airport_location": "Dallas",
414         "other_details": "International"
415     },
416     {
417         "_id": "ATL",
418         "airport_name": "Atlanta International Airport",
419         "airport_location": "Atlanta",
420         "other_details": "International"
421     }
422 ]
423
424
425 inserted_airport_data = airport_col.insert_many(airports)
426
427

```

```
428 #flights cost collection
429
430 flight_cost = [
431     {
432         "_id": {
433             "flight_number": "AA321",
434             "aircraft_type_code": "B747"
435         },
436         "valid_from_date": "2023-01-01",
437         "valid_to_date": "2025-01-01",
438         "flight_cost": 9845
439     },
440     {
441         "_id": {
442             "flight_number": "AA642",
443             "aircraft_type_code": "B360"
444         },
445         "valid_from_date": "2023-01-01",
446         "valid_to_date": "2025-01-01",
447         "flight_cost": 2478
448     },
449     {
450         "_id": {
451             "flight_number": "AA985",
452             "aircraft_type_code": "S823"
453         },
454         "valid_from_date": "2023-01-01",
455         "valid_to_date": "2025-01-01",
456         "flight_cost": 3675
457     }
458 ]
459
460
461
462 inserted_flight_cost_data = flight_cost_col.insert_many(flight_cost)
463
464
465
```



```

467
468 #flights schedule collection
469
470 flight_sched = [
471     {
472         "_id": {
473             "flight_number": "AA321",
474             "airline_code": "JK35748",
475             "usual_aircraft_type_code": "B747",
476             "origin_airport_code": "LFT",
477             "destination_airport_code": "JFK"
478         },
479         "departure_date_time": "2023-10-12T10:00:00",
480         "arrival_date_time": "2023-10-12T12:00:00"
481     },
482     {
483         "_id": {
484             "flight_number": "AA642",
485             "airline_code": "JK26747",
486             "usual_aircraft_type_code": "B360",
487             "origin_airport_code": "MSY",
488             "destination_airport_code": "DFW"
489         },
490         "departure_date_time": "2023-10-12T10:00:00",
491         "arrival_date_time": "2023-10-12T12:00:00"
492     },
493     {
494         "_id": {
495             "flight_number": "AA985",
496             "airline_code": "JK10832",
497             "usual_aircraft_type_code": "S823",
498             "origin_airport_code": "MSY",
499             "destination_airport_code": "ATL"
500         },
501         "departure_date_time": "2023-10-12T10:00:00",
502         "arrival_date_time": "2023-10-12T12:00:00"
503     }
504 ]
505
506
507
508 inserted_flight_sched_data = flight_sched_col.insert_many(flight_sched)
509
510
511
512 #ref calendar collection
513
514 ref_calendar = [
515     {
516         "_id": "765354",
517         "day_number": 4,
518         "business_day_yn": "09-04-2024"
519     },
520     {
521         "_id": "794524",
522         "day_number": 12,
523         "business_day_yn": "08-12-2024"
524     },
525     {
526         "_id": "165424",
527         "day_number": 9,
528         "business_day_yn": "07-09-2024"
529     }
530 ]
531
532 inserted_ref_cal_data = ref_cal_col.insert_many(ref_calendar)
533

```

Name	Type	Size	Value
agent_collection	str	12	travel agent
airport_collection	str	8	airports
airports	list	5	[{'_id':'LFT', 'airport_name':'Lafayette Regional', 'airport_location'...
client	mongo_client.MongoClient	1	MongoClient object of pymongo.mongo_client module
cust_itin_pipeline	list	5	[{'\$match':{'...}}, {'\$lookup':{'...}}, {'\$unwind':{'...}}, {'\$lookup':{'...
cust_itinerary	list	1	[{'_id':'7463826', 'first_name':'Mila', 'last_name':'Romanova', 'reser...
db	database.Database	1	Database object of pymongo.database module
delay_vs_timely	list	5	[{'_id':{'...'}, 'flight_number':'AA321', 'departure_time':'2024-01-01 1...
flight_cost	list	3	[{'_id':{'...'}, 'valid_from_date':'2023-01-01', 'valid_to_date':'2025-0...
flight_cost_collection	str	11	flight cost
flight_leg	list	3	[{'_id':'056432', 'flight_number':'AA321', 'airline_code':'LFT', 'orig...
flight_legs_collection	str	11	flight legs
flight_number	str	5	AA321
flight_sched	list	3	[{'_id':{'...'}, 'departure_date_time':'2023-10-12T10:00:00', 'arrival_d...
flight_sched_collection	str	15	flight schedule
flight_sched_pipeline	list	4	[{'\$lookup':{'...}}, {'\$lookup':{'...}}, {'\$project':{'...}}, {'\$sort':{'...
flight_schedules	list	3	[{'_id':{'...}}, {'_id':{'...}}, {'_id':{'...}}]
flight_status	command_cursor.CommandCursor	1	CommandCursor object of pymongo.command_cursor module
flights_at_jfk	command_cursor.CommandCursor	1	CommandCursor object of pymongo.command_cursor module
inserted_agent_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_airport_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_flight_cost_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_flight_leg_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_flight_sched_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_itin_leg_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_itin_res_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_passenger_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_ref_cal_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
inserted_res_payment_data	results.InsertManyResult	1	InsertManyResult object of pymongo.results module
itin_leg	list	5	[{'_id':{'...'}, 'flight_number':'AA321', 'airline_code':'LFT', 'origin_...
itin_leg_collection	str	14	itinerary legs
itin_res	list	5	[{'_id':'15457568', 'agent_id':'045231', 'passenger_id':'7463826', 're...
itin_res_collection	str	22	itinerary reservations
jfk_airport_code	str	3	JFK
jfk_flight	list	3	[{'_id':{'...'}, 'flight_number':'AA321', 'departure_airport':'Lafayette...

jfk_flight_pipeline	list	6	[{'_id': {'\$match': {...}}, {'\$lookup': {...}}, {'\$lookup': {...}}, {'\$unwind': '\$...'}]
new_database	database.Database	1	Database object of pymongo.database module
pass_collection	list	5	[{'_id': '7463826', 'first_name': 'Mila', 'second_name': 'Dina', 'last_name': ...}]
pass_on_flight	list	5	[{'_id': '7463826', 'first_name': 'Mila', 'last_name': 'Romanova', 'flight_number': ...}]
pass_on_flight_pipeline	list	7	[{'\$lookup': {...}}, {'\$unwind': {...}}, {'\$lookup': {...}}, {'\$unwind': {'\$flight_number': ...}}]
passenger_id	str	7	7463826
pipeline	list	2	[{'\$match': {...}}, {'\$group': {...}}]
project_2_database	str	7	Airline
ref_cal_collection	str	19	reference calendar
ref_calendar	list	3	[{'_id': '765354', 'day_number': 4, 'business_day_yn': '09-04-2024'}, {'_id': ...}]
res_pay	list	5	[{'_id': 'AV6345', 'pass_id': '7463826', 'flight_number': 'AA321', 'reservation_payment': ...}]
res_payment_collection	str	20	reservation payments
salesaa321	list	1	[{'_id': 'AA321', 'total_sales': 1797}]
time_check_pipeline	list	2	[{'\$match': {...}}, {'\$project': {...}}]
travel_agents	list	3	[{'_id': '045231', 'agent_name': 'Sue Jones', 'agent_details': 'Associate Agent', ...}]

- View his itinerary

```

534 # Customer view itinerary
535
536 # Define which passenger
537
538 passenger_id = "7463826"
539
540 # Pipeline to retrieve necessary information
541
542 cust_itin_pipeline = [
543     {
544         '$match': {
545             '_id': passenger_id
546         }
547     },
548     {
549         '$lookup': {
550             'from': 'itinerary reservations',
551             'localField': 'passenger_id',
552             'foreignField': 'passenger_id',
553             'as': 'res'
554         }
555     },
556     {
557         '$unwind': {
558             'path': '$reservations',
559             'preserveNullAndEmptyArrays': True
560         }
561     },
562     {
563         '$lookup': {
564             'from': 'itinerary legs',
565             'localField': 'res._id',
566             'foreignField': 'reservation_id',
567             'as': 'legs'
568         }
569     },
570     {
571         '$project': {
572             'first_name': 1,
573             'last_name': 1,
574             'reservations': {
575                 'reservation_id': '$reservations._id',
576                 'payment_amount': '$reservations.payment_amount',
577                 'origin': '$res.origin_airport_code',
578                 'destination': '$res.destination_airport_code',
579                 'date_reservation_made': '$res.date_reservation_made',
580                 'legs': '$legs'
581             }
582         }
583     }
584 ]
585
586 # Aggregate and list the information
587
588 cust_itinerary = list(db['passengers'].aggregate(cust_itin_pipeline))
589
590
591

```

cust_itinerary - List (1 element)

Index	Type	Size	Value
0	dict	4	{'_id': '7463826', 'first_name': 'Mila', 'last_name': 'Romanova', 'reserv ...

0 - Dictionary (4 elements)

Key	Type	Size	Value
_id	str	7	7463826
first_name	str	4	Mila
last_name	str	8	Romanova
reservations	dict	4	{'origin':[], 'destination':[], 'date_reservation_made':[], 'legs':[{. ...

reservations - Dictionary (4 elements)

Key	Type	Size	Value
date_reservation_made	list	0	[]
destination	list	0	[]
legs	list	5	[{'_id':{...}, 'flight_number':'AA321', 'airline_code':'LFT', 'origin_ ...
origin	list	0	[]

- **Get all customers who have seats reserved on a given flight.**

```

590 # Find all seats on a given flight
591
592 # Determine which flight to look up
593
594 flight_number = "A4321"
595
596 # Pipeline to find the names of passengers
597
598 pass_on_flight_pipeline = [
599     {
600         '$lookup': {
601             'from': 'reservation payments',
602             'localField': 'passenger_id',
603             'foreignField': 'pass_id',
604             'as': 'payment_info'
605         },
606     },
607     {
608         '$unwind': {
609             'path': '$payment_info',
610             'preserveNullAndEmptyArrays': True
611         },
612     },
613     {
614         '$lookup': {
615             'from': 'itinerary reservations',
616             'localField': 'passenger_id',
617             'foreignField': 'passenger_id',
618             'as': 'res_info'
619         },
620     },
621     {
622         '$unwind': {
623             'path': '$res_info',
624             'preserveNullAndEmptyArrays': True
625         },
626     },
627     {
628         '$lookup': {
629             'from': 'itinerary legs',
630             'localField': 'res_info_id',
631             'foreignField': 'reservation_id',
632             'as': 'itin_leg'
633         },
634     },
635     {
636         '$match': {
637             'itin_leg.flight_number': flight_number
638         },
639     },
640     {
641         '$project': {
642             'first_name': '$first_name',
643             'last_name': '$last_name',
644             'flight_number': '$itin_leg.flight_number'
645         },
646     },
647 ]
648
649 # Execute the aggregation
650 pass_on_flight = list(db['passengers'].aggregate(pass_on_flight_pipeline))
651
652

```

pass_on_flight - List (5 elements)



Index	Type	Size	Value
0	dict	4	{'_id': '7463826', 'first_name': 'Mila', 'last_name': 'Romanova', 'flight ...
1	dict	4	{'_id': '5885922', 'first_name': 'Laura', 'last_name': 'Juarez', 'flight_ ...
2	dict	4	{'_id': '9284293', 'first_name': 'Frank', 'last_name': 'Craft', 'flight_n ...
3	dict	4	{'_id': '7375758', 'first_name': 'Jian', 'last_name': 'Cho', 'flight_num ...
4	dict	4	{'_id': '2772849', 'first_name': 'Joe', 'last_name': 'Hue', 'flight_num ...

- Get all flights for a given airport.

```

653 # All flights from JFK
654
655 # Define the airport code for JFK
656 jfk_airport_code = 'JFK'
657
658 # Pipeline to retrieve flights from JFK
659
660 jfk_flight_pipeline = [
661     {
662         '$match': {
663             '$or': [
664                 {'origin_airport_code': jfk_airport_code},
665                 {'destination_airport_code': jfk_airport_code}
666             ]
667         },
668     },
669     {
670         '$lookup': {
671             'from': 'airports',
672             'localField': 'origin_airport_code',
673             'foreignField': '_id',
674             'as': 'origin_airport'
675         },
676     },
677     {
678         '$lookup': {
679             'from': 'airports',
680             'localField': 'destination_airport_code',
681             'foreignField': '_id',
682             'as': 'destination_airport'
683         },
684     },
685     {
686         '$unwind': '$origin_airport'
687     },
688     {
689         '$unwind': '$destination_airport'
690     },
691     {
692         '$project': {
693             'flight_number': '$flight_number',
694             'departure_airport': '$origin_airport.airport_name',
695             'arrival_airport': '$destination_airport.airport_name',
696             'departure_time': '$departure_date_time',
697             'arrival_time': '$arrival_date_time'
698         }
699     }
700 ]
701
702 # Execute the aggregation and list results
703
704 flights_at_jfk = db['itinerary Legs'].aggregate(jfk_flight_pipeline)
705
706 jfk_flight = list(flights_at_jfk)
707
708
709

```

jfk_flight - List (3 elements)



Index	Type	Size	Value
0	dict	6	{'_id': {'reservation_id': '15457568', 'leg_id': '056432'}, 'flight_numbe ...
1	dict	6	{'_id': {'reservation_id': '35635773', 'leg_id': '656833'}, 'flight_numbe ...
2	dict	6	{'_id': {'reservation_id': '74677424', 'leg_id': '056432'}, 'flight_numbe ...

0 - Dictionary (6 elements)

Key	Type	Size	Value
_id	dict	2	{'reservation_id': '15457568', 'leg_id': '056432'}
arrival_airport	str	23	John F. Kennedy Airport
arrival_time	str	19	2024-01-01 20:00:00
departure_airport	str	18	Lafayette Regional
departure_time	str	19	2024-01-01 17:00:00
flight_number	str	5	AA321

- View flight schedule.

```

709 # View flight schedules
710 # Pipeline to create a list of flight schedules
711
712 flight_sched_pipeline = [
713     {
714         '$lookup': {
715             'from': 'airports',
716             'localField': 'departure_airport_code',
717             'foreignField': 'airport_code',
718             'as': 'dep_apt'
719         },
720     },
721     {
722         '$lookup': {
723             'from': 'airports',
724             'localField': 'arrival_airport_code',
725             'foreignField': 'airport_code',
726             'as': 'arr_apt'
727         },
728     },
729     {
730         '$project': {
731             'flight_number': 1,
732             'departure_time': 1,
733             'arrival_time': 1,
734         },
735     },
736     {
737         '$sort': {
738             'departure_time': 1 # Sort by departure time
739         }
740     }
741 ]
742
743 # Aggregate and list the flight schedules
744
745 flight_schedules = list(db['flight_schedule'].aggregate(flight_sched_pipeline))
746
747
748

```

flight_schedules - List (3 elements)

Index	Type	Size	Value
0	dict	1	{'_id': {'flight_number': 'AA321', 'airline_code': 'JK35748', 'usual_airc ...
1	dict	1	{'_id': {'flight_number': 'AA642', 'airline_code': 'JK26747', 'usual_airc ...
2	dict	1	{'_id': {'flight_number': 'AA985', 'airline_code': 'JK10832', 'usual_airc ...

0 - Dictionary (3 elements)

Key	Type	Size	Value
_id	dict	5	{'flight_number': 'AA321', 'airline_code': 'JK35748', 'usual_aircraft_ty ...
arrival_date_time	str	19	2023-10-12T12:00:00
departure_date_time	str	19	2023-10-12T10:00:00

- Get all flights whose arrival and departure times are on time/delayed.

```

749 # Determine delayed or on time flights
750
751 # Pipeline to check flight timeliness
752
753 time_check_pipeline = [
754     {
755         '$match': {
756             '$or': [
757                 {'departure_efficiency': True}, # On-time
758                 {'departure_efficiency': False} # Delayed
759             ]
760         },
761     },
762     {
763         '$project': {
764             'flight_number': '$flight_number',
765             'departure_time': '$departure_date_time',
766             'arrival_time': '$arrival_date_time',
767             'status': {
768                 '$cond': {
769                     'if': {'$eq': ['$departure_efficiency', True]},
770                     'then': 'On Time',
771                     'else': 'Delayed'
772                 }
773             }
774         }
775     }
776 ]
777
778 # Execute the aggregation
779
780 flight_status = db['itinerary Legs'].aggregate(time_check_pipeline)
781
782 # List the results
783
784 delay_vs_timely = list(flight_status)
785

```

delay_vs_timely - List (5 elements)

Index	Type	Size	Value
0	dict	5	{'_id':{'reservation_id':'15457568', 'leg_id':'056432'}, 'flight_numbe ...
1	dict	5	{'_id':{'reservation_id':'24668346', 'leg_id':'056432'}, 'flight_numbe ...
2	dict	5	{'_id':{'reservation_id':'35635773', 'leg_id':'656833'}, 'flight_numbe ...
3	dict	5	{'_id':{'reservation_id':'36725613', 'leg_id':'345795'}, 'flight_numbe ...
4	dict	5	{'_id':{'reservation_id':'74677424', 'leg_id':'056432'}, 'flight_numbe ...

0 - Dictionary (5 elements)

Key	Type	Size	Value
_id	dict	2	{'reservation_id':'15457568', 'leg_id':'056432'}
arrival_time	str	19	2024-01-01 20:00:00
departure_time	str	19	2024-01-01 17:00:00
flight_number	str	5	AA321
status	str	7	Delayed

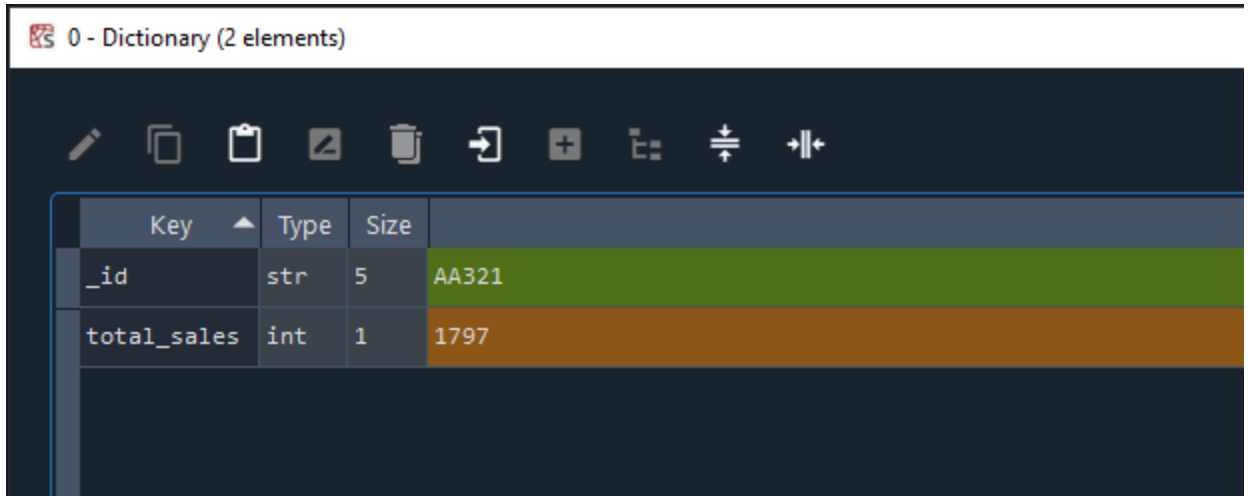
- Calculate total sales for a given flight.

```

788 # Calculate total sales on given flight
789
790 # Pipeline to sum sales for a given flight
791
792 sum_pipeline = [
793     {
794         '$match': {
795             'flight_number': 'AA321'
796         },
797     },
798     {
799         '$group': {
800             '_id': '$flight_number',
801             'total_sales': { '$sum': '$payment_amount' }
802         }
803     }
804 ]
805
806 # Execute the aggregation
807
808 salesaa321 = list(db['reservation_payments'].aggregate(sum_pipeline))
809

```

0 - Dictionary (2 elements)



Key	Type	Size	
_id	str	5	AA321
total_sales	int	1	1797

Part 2 – Discussion

Deciding between a relational database and a non-relational database for working with transactional data is a great undertaking for any organization that is scaling towards a larger size. The introduction of big data to companies who operate at high levels with thousands of millions of customers has given way to the possibility of gaining more insights into the performance of a business as well as into the needs of their stakeholders. Traditional relational database systems can no longer support the many types of data that are needed to gain insights into the full picture of modern operations.

The document model for a non-relational database is an excellent choice for the airline database system because it allows for flexibility in data storage and in the ability to modify the system as operations expand. Without the constraints of schema design in place as seen in relational databases, the airline can adapt more rapidly to the evolving needs of the organization (Parker et al, 2013, pp. 1-2). Systems like MongoDB also perform at a faster rate when handling larger data sets due to the fact that they use “memory mapped files” instead of retrieving from a disk like SQL databases (Parker et al, 2013, p. 5). An added benefit of using a non-relational

database over the decision of using a relational database is that there is the option to work with more advanced algorithms that can function with the database for better performance and analysis. This option is not available when strictly working with a database that only operates using SQL (Tamane, 2016, p. 1). A document database can also store valuable metadata which can be accessed via queries to provide deeper details into the main data being recorded in each collection in the database. A high-transaction system such as the one that would be operating for the airline company could benefit from being able to work with metadata that subsets the general information being stored for each transaction (Tamane, 2016, p. 2). Advanced algorithms being used can also work to provide predictions about future consumer behavior and other organizational performance metrics.

A disadvantage of using a document model non-relational databases is that there would be the need for conversion from the existing relational database. This can be a cumbersome task, but there are options available for a more smooth transition if the organization decided to take on this project. When considering the speed at which this could be done there is a benefit offered by the document model that offer enhanced performance during the conversion process of the data when compared to working with other options, such as the column-family structure, of storing data in non-relational databases (Kuszera et al., 2019, p. 462). The previously stored information could be converted to a document model non-relational databases through directed acyclic graphs (DAG) that would produce the appropriate JSON files needed (Kuszera et al., 2019, p. 457). But, the difficulty still exists for operating the new system due to the fact that there is a smaller community of individuals who have the skills needed to work with non-relational databases as opposed to the amount of people who can work with SQL based systems. An example of the difficulties that may arise would be that non-relational databases are more unstructured than their relational counterparts. The data needed to be analyzed may be spread across multiple documents that have no clear way to be joined easily which can cause issues

for staff and lead to productivity loss if the workers are not properly trained (Rhama et Al, 2019, p.45)

Privacy and security should be a concern for organizations that store personally identifiable information about their customers and other information about the organization that needs to be protected from bad actors. MongoDB has the capability to encrypt the data that is being collected while allowing for the data to be queried upon through a method referred to as 'Onions and Layers Concept (OLC)'. This method structures the encryption of particular information based on different levels of necessary encryption and allows for access to the encrypted data via queries while also providing different levels of access to unencrypted information (Almarwani et al., 2020, pp. 48-49). Having a protocol in place that allows for information to be worked with by employees without revealing the specific details such as credit card number or other personal information is a great benefit of working with MongoDB for a document model non-relational database.

Overall, document model non-relational databases provide many advantages over traditional relational database management systems and should be considered for operations if the resources are available. The benefits of having a database in place that can handle big data outweigh the disadvantages seen by trying to continue functioning with a relational database when scaling rapidly and needing to keep up with demands. Hiring staff with technical skills that reflect the current data landscape should be a priority of any business that strives to remain competitive in a global market. By not adapting to current trends, an organization puts themselves at risk of encountering major issues that can impact both productivity and profitability which is a strong reason to plan accordingly for future operational needs.

References

- Almarwani, M., Konev, B., & Lisitsa, A. (2020). Release-aware encryption adjustment query processing for document database. Proceedings of the 2020 4th International Conference on Cloud and Big Data Computing, 48–51. <https://doi.org/10.1145/3416921.3416933>
- Kuszera, E. M., Peres, L. M., & Fabro, M. D. (2019). Toward RDB to NoSQL. Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, 9714, 456–463. <https://doi.org/10.1145/3297280.3299734>
- Parker, Z., Poe, S., & Vrbsky, S. V. (2013). Comparing NoSQL mongodb to an SQL DB. Proceedings of the 51st ACM Southeast Conference. <https://doi.org/10.1145/2498328.2500047>
- Rahma S. Al Mahruqi, Manar H. Alalfi, and Thomas R . Dean. 2 019. A Semi-automated Framework for Migrating Web applications from SQL to Document Oriented NoSQL Database. In Proceedings of CASCON conference (CASCON '19). Toronto, Ontario, Canada, 10 pages.
- Tamane, S. (2016). Non-relational databases in Big Data. Proceedings of the Second International Conference on Information and Communication Technology for Competitive Strategies, 02, 1–4. <https://doi.org/10.1145/2905055.2905194>